

chapter_5_notes.pdf
PDF

我在复习计算机网络的课程。附件是第5章的slides,, 请你按照课件的思路, 详细的给我过。不要一下子输出所有, 而是输出一节解释, 让我理解一下, 并给我提问的机会。如果我没问题了再继续

已思考 5s >

好, 我们先从 **5.1 Introduction (控制平面概览 + routing/forwarding 的关系)** 这一节走一遍, 只讲这一节, 讲完我会停下来让你提问。  chapter_5_notes

5.1 Introduction: 这一章到底在讲什么

1) Network layer 两个核心工作: Forwarding vs Routing

课件一上来先把网络层分成两个“动作” (page 4):  chapter_5_notes

- **Forwarding (转发):** 发生在“每一台路由器本地”。
你拿到一个到达路由器的 packet, 看它 IP 目的地址, 然后查 **本地 forwarding table**, 决定从哪个 output link 发出去。
👉 关键: 它是“本地一步”的动作 (input → output), 偏执行层。
- **Routing (路由/选路):** 发生在“整个网络范围”。
决定从源到目的要走哪条路径 (end-to-end path), 也就是**如何产生/更新 forwarding table**。
👉 关键: 它是“全局规划”, 偏决策层。

课件在 page 8 用一张图把它们的关系讲得很直白:

routing algorithm 负责算端到端路径 → 结果落地成每台路由器的 local forwarding table → forwarding table 再指导每次转发。  chapter_5_notes

你可以把它类比成:

- routing = “地图导航在算路线”
- forwarding = “每个路口根据导航提示左转/右转”

2) 控制平面 (Control plane) 两种组织方式：传统 vs SDN

这章标题是 *Control Plane*，所以它关心的是：“**routing 这件事的决策/控制，网络到底怎么组织？**” (page 4-6)  chapter_5_notes

课件给了两种架构：

A. Per-router control (传统方式)

(page 5) 每台路由器里都有一套 routing algorithm 的组件。它们彼此交换信息、各自计算，然后各自生成自己的 forwarding table。  chapter_5_notes

- 优点：分布式、无需中心
- 直觉：像“每个人只跟邻居聊，然后自己推理出全局路线”

B. Logically centralized control (SDN 思路)

(page 6) 有一个“逻辑上中心化”的 controller (可以物理上远程)，它和每台路由器上的 control agent 交互，**统一计算并下发 forwarding table**。  chapter_5_notes

- 注意关键词是 *logically centralized*：逻辑上像一个大脑，但实际实现可以是集群、分布式容错
- 直觉：像“有一个调度中心给每个路口下发交通指令”

这一节的目的其实是：先把“控制平面是什么”立住，后面 5.2 才开始讲传统路由算法 (LS/DV)，5.5 才讲 SDN control plane 的细节。

3) 路由问题的抽象：图 (Graph abstraction)

从 page 9-10 开始，课件把网络抽象成图 $G = (N, E)$ ：  chapter_5_notes

- N ：路由器节点集合 (u, v, w, x, y, z)
 - E ：链路集合 (节点间有边)
 - 每条边有 **cost** $c(x, x')$ ，路径总代价是沿途边的 cost 之和
- 核心问题：**least-cost path** (例如 u 到 z 的最小代价路径) 怎么找？——这就是 routing algorithm 要解决的。  chapter_5_notes

同时课件提醒：cost 可以代表很多东西 (page 10)：

- 恒等于 1 (跳数)
 - 和带宽成反比
 - 和拥塞程度相关
- 这也为后面讲“动态路由、震荡”等现象埋伏笔。

你现在需要确认的 3 个点 (你回答我就行)

1. 你能用一句话说清 **routing vs forwarding** 的区别吗？（一个是算路，一个是按表转发）
2. 传统 per-router control 和 SDN centralized control 的差别，你脑子里有画面了吗？
3. 图抽象里：**节点、边、cost、path cost** 分别对应网络里的什么？

你回答/提问后，如果这一节没问题，我们就继续下一节 **5.2 Routing Protocols: Link State vs Distance Vector** 的分类与直觉（也按课件一节一节来）。

来源



1. routing是算路径，forwarding是按表转发
2. 传统的靠邻居之间彼此通信确定路径，sdn是靠逻辑大脑算好表直接给所有人应用
3. 节点对应路由器，边对应链路集合，cost代表cost，pathcost代表每条路的cost综合

很好，这三点都对上了。我们继续下一节：**5.2 Routing Protocols（路由协议分类 + 两大类算法直觉）**。  chapter_5_notes

5.2 Routing Protocols：路由协议在“控制平面”里干啥

这一节的核心是：路由协议就是让路由器们获得“到各目的地应该走哪条下一跳”的信息，从而形成/更新 forwarding table。（page 11）  chapter_5_notes

1) Routing protocol 的基本输出：forwarding table

课件强调：每个路由器最终要有一张表，典型就是

- 目的前缀（destination）
- 下一跳（next hop / outgoing interface）

Routing algorithm/Protocol 负责把这张表“算出来并持续更新”。  chapter_5_notes

2) 两大类：Link-State (LS) vs Distance-Vector (DV)

课件把“分布式算最短路”分成两派（page 12）：  chapter_5_notes

A) Link-State（链路状态）——“先让大家都知道全图，再各自算最短路”

直觉：

1. 每个路由器先知道自己直接相连链路的 cost
2. 通过 **flooding**（泛洪）把“我这条链路现在什么状态/代价”广播到全网
3. 于是每个路由器最后都拥有一份（近似）一致的全网拓扑+链路代价图
4. 然后每台路由器在本地跑 **Dijkstra** 算自己到所有目的地的最短路径树（Shortest Path Tree），填 forwarding table

你可以记成一句：**LS = 先同步全局信息，再本地计算。**  chapter_5_notes

B) Distance-Vector（距离向量）——“不需要全图，只跟邻居反复交换‘到目的地我有多远’”

直觉：

1. 每个路由器维护一个向量：到每个目的地 d 的当前估计距离 $D_x(d)$
2. 只和邻居交换各自的向量（而不是交换全拓扑）
3. 根据 Bellman-Ford 思想迭代更新：
“我到 d 的最好距离 = $\min(\text{到某邻居} + \text{邻居到 } d \text{ 的距离})$ ”
4. 反复交换直到收敛

一句话：**DV = 只看邻居的‘报价’，不断讨价还价直到稳定。**  chapter_5_notes

3) 这一节你现在应该形成的“对比感”

你先不用背细节，先抓住两种风格的差异：

- **信息范围**：LS 让大家都知道“全局拓扑”；DV 只知道“邻居的距离向量”
- **通信方式**：LS 典型是 flooding（扩散全网）；DV 是邻居之间周期/触发交换
- **计算位置**：LS 在本地跑 Dijkstra（有全图）；DV 不直接有全图，通过迭代逼近最短路径
- **典型风险直觉**：DV 更容易出现“坏消息传得慢/数到无穷”（后面会讲）；LS 更像“全网状态同步成本更高”（泛洪开销、状态一致性）  chapter_5_notes

轮到你：用一句话回答我这两个小问题

1. LS 为什么需要 flooding？它 flood 的到底是什么信息？
2. DV 为什么不需要全网拓扑也能算路？它反复交换的是什么？

你答完我们就进入下一小节：**5.3 Link-State Routing Algorithm（Dijkstra + SPT + 例题表格）**，按课件一步步推。

来源



1. 将信息散步到全部网络，从源头路由器开始flood路径上的cost和
2. 通过邻居的最小距离更新自己的

已思考 49s >

你这两题的方向都对，我帮你把措辞“卡准”一下，然后进入 **5.3 Link-State Routing Algorithm (Dijkstra)**，只讲这一节，讲完我停下来让你问。 chapter_5_notes

先纠正/补全你刚才两点

1. LS 为什么需要 flooding? flood 的是什么?

更精确地说：flood 的不是“路径上的 cost”，而是**每个路由器自己的链路状态信息** (link-state advertisement)：

“我是谁 + 我连着哪些邻居 + 到每个邻居这条链路的 cost”。把这些信息扩散到全网后，大家就都能拼出同一张“全网拓扑图 + 链路代价”。 chapter_5_notes

2. DV 为什么不需要全拓扑也能算路? 交换什么?

对：靠“邻居的最小距离更新自己的”。更精确：DV 反复和邻居交换的是 **distance vector**（到各目的地的当前估计距离表），并用

$$D(v) \leftarrow \min(D(v), D(w) + c(w, v))$$

这种 Bellman-Ford 风格规则迭代到收敛。 chapter_5_notes

5.3 Link-State Routing Algorithm: Dijkstra 怎么算出 forwarding table

1) 前提 (Link-State 的关键假设)

Dijkstra 这套在 LS 里能跑起来的原因是：“网络拓扑 & 链路代价对所有节点都已知”（通过 link-state broadcast / flooding 达成）。所以每个路由器都能在本地跑同一个最短路计算。

chapter_5_notes

2) Dijkstra 的三个符号：N'、D(·)、p(·)

课件的记法是（你一定要把这三个记法吃透）： chapter_5_notes

- **N'**: 已经“确定最短路”的节点集合 (settled set)。在任何时刻, N' 里的节点, 它们从源点 u 出发的最短路已经不会再变了。
- **D(v)**: 当前从源点 u 到 v 的最小“暂定”代价 (tentative distance)。
- **p(v)**: 让 $D(v)$ 取到当前最小值时, v 的前驱节点 (parent / predecessor)。它用于最后回溯出路径, 从而决定 next hop。

直觉一句话:

N' 是“已经板上钉钉的最短路点”; **D** 是“候选报价”; **p** 是“你这报价是从谁那儿继承来的”。

3) 算法流程 (你要会手推)

课件给的伪代码核心是:  chapter_5_notes

- 初始化: $N' = \{u\}$; 对每个 v : 若 v 与 u 相邻, $D(v) = c(u, v)$, 否则 $D(v) = \infty$
- 循环:
 1. 在所有 **不在 N'** 的节点里找 D 最小的 w
 2. 把 w 加进 N' (表示: $u \rightarrow w$ 的最短路现在“确定”了)
 3. 用 w 去松弛它的邻居 v :

$$D(v) = \min(D(v), D(w) + c(w, v))$$

若更新成功, 就把 $p(v) = w$

最关键的理解点:

每次挑 D 最小的 w 加入 N' , 是因为此时任何“绕路再回来”的路径都只会更贵, 所以 w 的最短路不会再被改写。

4) 用课件例子感受“表格怎么填”

课件给了一个从 u 出发的例子表 (Step 0, 1, 2, ...), 你只要看懂一个节奏就够了:

 chapter_5_notes

- Step 0: $N' = \{u\}$, 先把 u 的直接邻居距离写上 (比如 $D(x) = 1, u$; $D(v) = 2, u$; $D(w) = 5, u \dots$)
- Step 1: 挑最小的 (这里是 x), 加入 $N' \rightarrow$ 用 x 去更新它的邻居 (比如把 $D(y)$ 从 ∞ 更新成 $2, x$; 把 $D(w)$ 从 $5, u$ 更新成 $4, x \dots$)
- Step 2: 再挑最小的 (比如 y), 加入 $N' \rightarrow$ 用 y 继续更新 (比如 $D(w)$ 可能被更新成 $3, y$; $D(z)$ 被更新成 $4, y \dots$)
- ...直到所有点都进 N'

最后你得到的是一棵从 u 出发的 **shortest-path tree (SPT)**，以及每个目的地的 $p(\cdot)$ 链。

 chapter_5_notes

5) SPT 怎么变成 forwarding table (“下一跳”怎么来)

这是很多人做题最容易卡的点：

对任意目的地 d ，你从 d 开始沿着 $p(d), p(p(d))\dots$ 一路回溯到 u ，得到完整路径。

next hop 就是这条路径里 “ u 后面的第一个节点”。

课件在例子里给了 u 的 resulting forwarding table (destination $\rightarrow$ outgoing link)。

 chapter_5_notes

6) 讨论：复杂度 & 可能的 oscillation

- 复杂度：朴素实现每轮要扫描所有未加入 N' 的节点找最小 D ，整体 $O(n^2)$ ；也可以用更高效的数据结构做到 $O(n \log n)$ 。  chapter_5_notes
- oscillations possible**：如果链路 cost 取决于拥塞/流量 (cost=carried traffic 这类“敏感”定义)，路由一变 $\rightarrow$ 流量重分配 $\rightarrow$ cost 又变 $\rightarrow$ 路由再变，可能出现来回震荡 (route flapping)。课件用一个小网络示意了这种现象。  chapter_5_notes

现在轮到你（两道很短的确认题）

- 在 Dijkstra 里， N' 的语义用你的话再说一遍：为什么“加进 N' 之后就不会再变”？
- 假设你已经有了每个节点的 $p(\cdot)$ ，你能用一句话说清：怎么从 $p(\cdot)$ 得到 forwarding table 的 next hop？

你答完/或者你想我带你手推一轮课件表格（比如从 Step 0 到 Step 2），我再继续下一节（通常会接着讲 DV / 或者 LS 的更多讨论，按你的课件顺序走）。

来源

     

pause, ls的flooding是每个路由器都要flood一次，还是一共开始一次flood就可以讲路过的路由器的情况都记录进来

是每个路由器都要（周期性/事件触发地）发自己的 link-state 信息，不是“从某个源头开始一次 flood 就把沿途路由器的情况顺便收集完”。

更具体点：

- **flood 的内容**是“我这台路由器的邻居列表 + 到每个邻居的链路 cost（以及序列号、老化时间等）”。这是一条 **LSA/LSP**。
- **谁来发**？全网每台路由器都会生成并发出自己的 LSA（在启动时、周期性刷新、或链路状态变化时）。
- **怎么扩散**？一台路由器发出后，它的邻居收到会继续转发给自己的其他邻居（除来路/或按规则抑制重复），所以这条 LSA 会“泛洪”到全网。
- **路由器怎么“记录”**？每台路由器会把收到的各个路由器的 LSA 存起来，拼成一份全网拓扑数据库（LSDB），然后在本地跑 Dijkstra。

你提到的“沿途把路过的路由器情况都记录进来”更像是 **traceroute/探测路径** 或者“收集式遍历”，但 LS flooding 不是收集，而是公告（advertise）+扩散（flood）：信息从每个节点向外散播。

你再确认一句：如果只有一台路由器发一次 flood，你觉得它能知道“别的路由器连着谁、cost 是多少”吗？（这就是为什么要“每台都公告自己”）



[回到dijkstra](#)

已思考 23s >

好的，回到 **5.3 Link-State Routing: Dijkstra**。我们就按课件的这一小节走：先把符号吃透 → 手推例子 Step 0~2 → 我停下来让你做下一步。  chapter_5_notes

1) 三个符号再“钉死”一次：N'、D(·)、p(·)

课件定义是：  chapter_5_notes

- **N'**：从源点 u 出发，“最短路已经确定”的节点集合（definitively known）。
- **D(v)**：当前 $u \rightarrow v$ 的暂定最小代价（tentative distance）。
- **p(v)**：当前这条最小代价路径里， v 的前驱节点（predecessor）。

算法伪代码核心就是：每轮选一个 **不在 N' 且 D 最小的 w** ，加进 N' ，然后用 w 去“松弛”它的邻居：

$$D(v) \leftarrow \min(D(v), D(w) + c(w, v))$$

📄 chapter_5_notes

2) 例子手推（课件表格 Step 0 → Step 2）

我们直接照课件的表（从源点 u 开始）。 📄 chapter_5_notes

Step 0 (Initialization)

- $N' = \{u\}$
- 已知 u 的直接邻居：
 - $D(x)=1, p(x)=u$
 - $D(v)=2, p(v)=u$
 - $D(w)=5, p(w)=u$
 - $D(y)=\infty, D(z)=\infty$

这些就是课件表 Step 0 的那一行。 📄 chapter_5_notes

Step 1（选最小 D 的节点加入 N' ）

现在 N' 外面最小的是 x ($D=1$)，所以：

- 把 x 加入 $N' \rightarrow N' = \{u, x\}$
- 用 x 去更新它的邻居（松弛）：
 - y ：原来 ∞ ，现在可以走 $u \rightarrow x \rightarrow y$ ，代价 $1+1=2$
 $\Rightarrow D(y)=2, p(y)=x$
 - w ：原来 5 ，现在走 $u \rightarrow x \rightarrow w$ ，代价 $1+3=4$
 $\Rightarrow D(w)=4, p(w)=x$
 - v ：表里保持 $D(v)=2$ （说明走 x 并没有更便宜）

这一整步对应课件 Step 1 那一列更新结果。 📄 chapter_5_notes

Step 2（再选最小 D ）

此时 N' 外面最小的有 v 和 y ，它们都 $D=2$ 。课件的选择顺序是先选 y ： 📄 chapter_5_notes

- $N' = \{u, x, y\}$
- 用 y 去松弛邻居：
 - w ：原来 $D(w)=4$ （经 x ），现在走 $u \rightarrow x \rightarrow y \rightarrow w$ ，代价 $2+1=3$
 $\Rightarrow D(w)=3, p(w)=y$

- z : 原来 ∞ , 现在走 $u \rightarrow x \rightarrow y \rightarrow z$, 代价 $2+2=4$
 $\Rightarrow D(z)=4, p(z)=y$
- v : 仍是 2 (不变)

到这里, 你应该能把课件表格的 Step 0/1/2 全看懂了。  chapter_5_notes

3) 你刚才问的两个“关键理解点”, 放在 Dijkstra 里怎么说

A) 为什么加入 N' 后就不会再变?

因为每一轮挑的 w 是 **当前所有 N' 外节点里 D 最小的**。任何“绕一圈再到 w ”的路径, 必然先到某个还不在 N' 的节点, 而那些节点的暂定距离都 $\geq D(w)$, 所以绕路只会更贵, 不可能把 w 的最短路再压低。

(你现在先用这个直觉记住就行: **最小者先确定, 确定后不回头**。)  chapter_5_notes

B) 有了 $p(\cdot)$ 怎么得到 forwarding table 的 next hop?

对每个目的地 d : 沿着 $p(d) \rightarrow p(p(d)) \rightarrow \dots$ 一直回溯到 u , 得到整条最短路径。

next hop = 这条路径里紧跟在 u 后面的第一个节点。课件在例子后面就给了 u 的 forwarding table (destination $\rightarrow$ link)。  chapter_5_notes

轮到你: 请你继续推 Step 3 (只问两个小问题)

在 Step 2 结束后 (也就是现在 $N'=\{u,x,y\}$, 且 $D(v)=2, D(w)=3, D(z)=4$):

1. 下一轮应该把谁加入 N' ? ($v / w / z$)
2. 加进去后, **哪些 $D(\cdot)$ 可能会被更新**? 你不需要算出所有数值, 先说“可能更新谁、为什么”。

你答完我再接着把 Step 3~5 完整推完, 并把最终的 shortest-path tree 和 forwarding table 怎么读出来讲清楚。

来源

     

问: dijkstra是确定起点和终点算cost最小路径的吗; N' 中维护的串是有顺序的吗

两个问题都问到点上了。

1) Dijkstra 是“起点到终点”还是“起点到所有点”？

在课件这章 (Link-State routing) 里, Dijkstra 用的是 **single-source shortest paths**: 给定一个源路由器 u , 算出 u 到网络里所有节点的最小代价路径 (从而填出 u 的 forwarding table)。
 chapter_5_notes

但如果你只关心某一个目的地 z , 也可以把它当成“起点到终点”:

- 运行过程中一旦 z 被选中加入 N' (也就是它的最短路已经确定), 就可以提前停, 不必把全网都算完。

2) N' 里维护的“串”有顺序吗？

概念上 N' 是集合 (set), 没有顺序: 它表示“哪些节点的最短路已经确定”。

 chapter_5_notes

但在执行算法/做表格题时, 会有一个“加入顺序”:

- 每一轮都会把当前 $D(\cdot)$ 最小的那个节点加入 N' , 所以你在表格里看到的 N' 常写成 “ $u, x, y, \dots$ ” 这种序列——那只是为了记录过程。
- 如果出现并列最小 (tie), 选哪个先加入可能会影响 $p(\cdot)$ 的具体前驱 (尤其是有多条等价最短路时), 但不会把最短距离算错; 最后的 forwarding table 在等价最短路的情况下可能有不同但都正确的 next hop。

你要不要我用一句话把“什么时候可以提前停止”和“tie 的处理对答案影响”总结成做题规则, 方便你考试直接套?

来源



N' 中加入一个节点的时候, 表明的信息知识: 我知道了从源头到这个新节点的最短路径怎么走, 但是具体怎么走还是不知道是吗

当一个节点 w 被加入 N' 的那一刻, 表示两件事都“定了”:

1. 最短距离定了: $D(w)$ 就是从源点 u 到 w 的最小 cost, 以后不会再变。
2. 路径怎么走也定了 (可恢复): 虽然你可能没有把整条路径一条条写出来, 但你已经维护了 前驱指针 $p(\cdot)$ 。
 - $p(w)$ 告诉你 w 的前一个节点是谁
 - 再沿着 $p(p(w))$ 、 $p(p(p(w)))$... 一直回溯到 u , 就能恢复出完整路径

所以不是“还不知道怎么走”, 而是信息已经在 $p(\cdot)$ 里了, 只是你还没把它显式展开。

小检查: 如果现在 $w \in N'$, 你觉得从 w 往回追 $p(w)$ 会发生什么? (最终一定会追到 u , 否则说明 p 记录有问题。)



所以一个单源头的dijkstra维护的是: N' , 和每个 n' 里的人的 p 和 D 对吗

对, 单源 Dijkstra 在运行时核心就维护这三样 (再加一个“未确定集合”的概念):

- N' : 已经“确定最短路”的节点集合
- 对每个节点 v :
 - $D(v)$: 当前从源 u 到 v 的暂定最小距离 (对 $v \in N'$ 时它已经最终确定)
 - $p(v)$: 让当前 $D(v)$ 取到最小的前驱节点 (用来回溯出路径/决定 next hop)

实现上通常还会有一个“未确定节点的优先队列/集合” (根据 $D(v)$ 取最小), 但逻辑上就是你说的: 集合 N' + 每个节点的 D 和 p 。



轮到你: 请你继续推 Step 3 (只问两个小问题)

在 Step 2 结束后 (也就是现在 $N'=\{u,x,y\}$, 且 $D(v)=2$, $D(w)=3$, $D(z)=4$):

1. 下一轮应该把谁加入 N' ? ($v/w/z$)
2. 加进去后, 哪些 $D(\cdot)$ 可能会被更新? 你不需要算出所有数值, 先说“可能更新谁、为什么”。

你答完我再接着把 Step 3~5 完整推完, 并把最终的 shortest-path tree 和 forwarding table 怎么读出来讲清

楚。
v, 没有更新

已思考 15s >

对的：**Step 3** 选 v, 而且不会更新任何 $D(\cdot)$ ——因为从 v 去更新 w 的候选代价是 $2 + 3 = 5$, 比当前 $D(w) = 3$ 差。  chapter_5_notes

下面我把 **Step 3~5** 按课件表格完整推完：  chapter_5_notes

Step 3 (你刚做的)

- 选最小的: $D(v) = 2$ (v 不在 N')
 - 加入: $N' = \{u, x, y, v\}$
 - 松弛 v 的邻居 (主要看 w):
 - 候选 $u \rightarrow v \rightarrow w : 2 + 3 = 5$, 当前 $D(w) = 3 \Rightarrow$ 不更新
- 所以这一轮 所有 D 都不变。  chapter_5_notes

Step 4

此时未加入的还有 w、z, 且

- $D(w) = 3$ (通过 y), $D(z) = 4$ (通过 y)  chapter_5_notes

所以:

- 选 w, 加入: $N' = \{u, x, y, v, w\}$
 - 松弛 w 的邻居 (主要看 z):
 - 候选 $u \rightarrow \dots \rightarrow w \rightarrow z : 3 + 5 = 8$, 当前 $D(z) = 4 \Rightarrow$ 不更新
- 仍然 D 不变。  chapter_5_notes

Step 5

- 只剩 z, 加入: $N' = \{u, x, y, v, w, z\}$ 完成。  chapter_5_notes

最终结果怎么读出来 (SPT + forwarding table)

从课件给的最终 shortest-path tree 和 u 的 forwarding table 结论是:  chapter_5_notes

- 到 v: 走 (u, v)
- 到 x: 走 (u, x)
- 到 y: 最短路是 $u \rightarrow x \rightarrow y$, 所以 next hop 是 x

- 到 w : 最短路是 $u \rightarrow x \rightarrow y \rightarrow w$, 所以 **next hop 还是 x**
- 到 z : 最短路是 $u \rightarrow x \rightarrow y \rightarrow z$, 所以 **next hop 还是 x**  chapter_5_notes

轮到你（两问，确认你真的会“读表→读 next hop”）

1. 你能从 $p(z) = y$ 、 $p(y) = x$ 、 $p(x) = u$ 推出: u 到 z 的 **next hop** 是谁吗?
2. 为什么到 w 的 next hop 不是 y , 而是 x ? (提示: next hop 定义是“紧跟 u 后面的第一个节点”)

你答完, 如果没问题, 我们就继续下一节 **Distance Vector / Bellman-Ford** (课件 5-19 开始)。  chapter_5_notes

来源



能, 是 x , 因为 u 不跟 y 直连?

对, **next hop 是 x** 。

但你给的理由要稍微修正一下: 不只是“ u 不跟 y 直连”, 而是 **next hop 的定义** 决定的:

- 我们已经知道最短路是 $u \rightarrow x \rightarrow y \rightarrow z$ (从 $p(z) = y, p(y) = x, p(x) = u$ 回溯得到)。
- **next hop** 永远取这条路径里 **紧跟在 u 后面的第一个节点**, 所以是 x 。

“ u 不跟 y 直连”只是一个辅助直觉: 如果 next hop 是 y , 那就意味着第一跳要从 u 直接到 y ; 既然没有这条链路 (或不在最短路里), 当然不可能。

再回答你上一轮我问的第 2 题:

到 w 的 next hop 不是 y , 而是 x , 原因同样是定义: 即使路径里经过 y ($u \rightarrow x \rightarrow y \rightarrow w$), 第一跳仍然是 $u \rightarrow x$ 。

如果你这块完全没问题, 我们就进入下一节 **5.4 Distance Vector (DV)**: Bellman-Ford 更新式、邻居交换、收敛、以及最经典的 **count-to-infinity** 为什么会发生。



继续

已思考 6s >

好，我们进入下一节：**Distance Vector (DV) 算法本体：Bellman-Ford + 每个节点维护什么 + 怎么交换怎么更新**（大概对应 5-19~5-23）。 [chapter_5_notes](#)

5-19: DV 的核心公式 (Bellman-Ford)

DV 的一句话核心就是（课件原式）：对任意目的地 y ，节点 x 到 y 的最短路代价满足

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

其中 v 枚举 x 的邻居； $c(x, v)$ 是 x 到邻居 v 的链路代价； $d_v(y)$ 是“邻居 v 到目的地 y 的最短代价”。 [chapter_5_notes](#)

直觉：“我到 y 最便宜的走法，一定是：先走到某个邻居 v ，再从 v 去 y ；所以我只要比较所有邻居的报价就行。” [chapter_5_notes](#)

5-20: Bellman-Ford 小例子怎么读

课件给了一个例子算 $d_u(z)$ ：已知邻居们到 z 的距离 ($d_v(z), d_x(z), d_w(z)$)， u 只要把“到邻居的成本 + 邻居到 z 的成本”算出来取最小即可：

$$d_u(z) = \min\{c(u, v) + d_v(z), c(u, x) + d_x(z), c(u, w) + d_w(z)\}$$

然后取到最小的那个邻居，就是 **next hop**（用于 forwarding table）。 [chapter_5_notes](#)

你可以把它当成：邻居在报“到 z 的总价”，你只需要加上“我到他那一步”再比价。

5-21: DV 算法里，每个节点到底维护什么？

课件用 $D_x(y)$ 表示“ x 对到 y 的最短代价的估计值”。 x 维护一整条向量：

$$D_x = [D_x(y) : y \in N]$$

并且 x 还会保存 每个邻居 v 的距离向量 D_v （因为邻居会发给它）。 [chapter_5_notes](#)

所以，在实现层面你可以理解成 x 有两类数据：

1. **本地已知**：到每个邻居 v 的链路代价 $c(x, v)$ （这是真实直接测到的）
2. **从邻居学到**：邻居 v 声称的“ v 到所有目的地 y 的代价” $D_v(y)$  chapter_5_notes

5-22：DV 的“交换 + 更新”机制

DV 的关键机制是：**每隔一段时间（或有变化时）节点把自己的 distance vector 发给邻居**。当 x 收到邻居 v 的新 DV 后，就对所有目的地 y 做一次更新：

$$D_x(y) \leftarrow \min_v \{c(x, v) + D_v(y)\}$$

（注意这里右边用的是邻居发来的估计 $D_v(y)$ ，不是“真实最短路”）。  chapter_5_notes

课件还说在一些自然条件下会收敛到真实最短路代价。  chapter_5_notes

5-23：DV 的三个“行为特征”（考试常考描述题）

DV 是：  chapter_5_notes

- **iterative（迭代）**：一次更新只用邻居当前的估计，不是一步到位算完。
 - **asynchronous（异步）**：不需要全网统一时钟，可能是“收到邻居更新”就触发一次本地更新。
 - **distributed（分布式）**：只和邻居通信；并且通常“当自己的 DV 发生变化时才通知邻居”，邻居再传播。  chapter_5_notes
-

现在停一下：你确认两个点（你答完我再继续讲 5-24~5-27 的表格演化 + good news / count-to-infinity）

1. 在 DV 里， x 为什么只需要邻居的 DV 就能更新自己？用一句话解释 Bellman-Ford 的直觉。
2. “next hop”在 DV 里怎么决定？（结合 5-20：哪个邻居对应最小值就选哪个）

 chapter_5_notes

来源

     ...

5-21: DV 算法里, 每个节点到底维护什么?

这个问题我有疑问: 维护一整条向量, 我想当然包括出发点
到所有邻居 (如果有 x 个邻居, 那就 x 个 cost) 的 cost, 以及
什么

你这个疑问非常关键: **DV 里“distance vector”本身不是“到邻居的链路 cost 列表”, 而是
“到所有目的地的当前最小代价估计”。**到邻居的 cost 属于另一类本地信息。

按课件的思路, 把一个节点 x 维护的东西分成三块最清晰:  chapter_5_notes

1) 本地链路信息: 到每个邻居的 cost

这是你说的那一部分: 如果 x 有 k 个邻居, 就有 k 个数

$$c(x, v_1), c(x, v_2), \dots, c(x, v_k)$$

这叫 **link cost to neighbors**, 是“我到邻居一跳”的真实代价 (直接测/配置得来)。

✅ 这部分一定要存, 但它不等于 distance vector。

2) 自己的 distance vector: 到所有目的地的估计 $D_x(\cdot)$

DV 的“向量”是:

$$D_x = [D_x(y) : y \in N]$$

也就是说, 对网络里每个可能的目的地 y (通常是每个路由器/每个前缀), 我都存一个“目前估计的最小代价”。

- 对自己: $D_x(x) = 0$
- 对直接邻居 v : 初始化常是 $D_x(v) = c(x, v)$
- 对非邻居: 初始化为 ∞ , 之后靠迭代变小/变大

✅ 这才是 **distance vector** 的本体: “我到各目的地有多远 (代价多少)”。

 chapter_5_notes

3) 路由选择信息: 对每个目的地的 next hop (以及可选的邻居向量缓存)

为了真正转发, 你还需要:

- **nextHop_x(y)**: 到目的地 y 走哪个邻居 (就是让 **min** 取到最小的那个邻居)

另外, 为了执行更新式

$$D_x(y) = \min_{v \in \text{Neigh}(x)} \{c(x, v) + D_v(y)\}$$

实现时通常还会缓存“从每个邻居 v 收到的 $D_v(\cdot)$ ” (不缓存也行, 但收到就更新更方便)。

📄 chapter_5_notes

你可以用一句话记住

- **链路表**: 我到邻居的真实一跳 cost (只对邻居, 数量=k)
- **距离向量**: 我到所有目的地的估计 cost (对全网目的地, 数量 $\approx |N|$ 或前缀数)
- **路由表/下一跳**: 每个目的地选哪个邻居实现这个最小值

给你一个很小的自测题 (你答一句就行):

如果 x 有 3 个邻居, 但网络里有 20 个路由器, 那么 x 的 **distance vector** 长度更接近 3 还是 20? 为什么?

来源

📄 🍌 🗨️ ⬆️ ↺️ ...

现在停一下: 你确认两个点 (你答完我再继续讲 5-24~5-27 的表格演化 + good news / count-to-infinity)

1. 在 DV 里, x 为什么只需要邻居的 DV 就能更新自己?
用一句话解释 Bellman-Ford 的直觉。

2. “next hop” 在 DV 里怎么决定? (结合 5-20: 哪个邻居对应最小值就选哪个)

1. 因为 x 到目的地必须通过某个邻居, 再加上自己与邻居的路径的 cost 就是全剧最小 cost (当前视角下) 2. 加上自己与邻居的 cost 之后, 哪个邻居的出路 cost 最小就是全剧最小
给你一个很小的自测题 (你答一句就行):

如果 x 有 3 个邻居, 但网络里有 20 个路由器, 那么 x 的 **distance vector** 长度更接近 3 还是 20? 为什么?

20

已思考 33s >

你这两点都答得对，而且措辞已经很“DV味儿”了：**我到目的地 = 先到某个邻居 + 邻居到目的地**；next hop 就是让这个和最小的邻居。 [chapter_5_notes](#)

下面我们继续 5-24~5-27：先把“距离表（distance table）怎么长、怎么更新”讲透，再讲 **good news travels fast / count-to-infinity**。 [chapter_5_notes](#)

5-24：DV 的“距离表”到底是什么样（以 node x 为例）

课件这里用 3 个节点 **x-y-z** 的小网络来演示（链路代价大致是：x-y=2, y-z=1, x-z=7）。 [chapter_5_notes](#)

1) x 维护的不是一张“邻居表”，而是一张“候选路径报价表”

对 node x 来说，它的邻居是 y 和 z，所以 x 的距离表可以理解成：

- **行 (from)**：我第一跳走哪个邻居 (y 或 z)
- **列 (cost to)**：我要去哪个目的地 (x、y、z)
- **表项 (row=v, col=dest=y)**：代表“我先到 v，再从 v 去 dest 的总代价”

$$\text{entry}(v, \text{dest}) = c(x, v) + D_v(\text{dest})$$

然后 x 自己的 distance vector 是把每一列取最小：

$$D_x(\text{dest}) = \min_{v \in \{y, z\}} \{c(x, v) + D_v(\text{dest})\}$$

这正是课件 5-24 的计算式。 [chapter_5_notes](#)

2) 按 5-24 的那两行计算，x 很快得到更短的 x→z

课件直接算了：

- $D_x(y) = \min\{2 + 0, 7 + 1\} = 2$
- $D_x(z) = \min\{2 + 1, 7 + 0\} = 3$

所以虽然 x 和 z 直连是 7，但通过 y 变成 **2+1=3**，这就是 DV “迭代交换邻居信息后发现更短路”的核心直觉。 [chapter_5_notes](#)

5-25：时间推进时发生了什么（“交换→更新→收敛”）

这一页在做的事就是重复一句话：**谁的 DV 变了就发给邻居，邻居收到就用 BF 式更新，最后会收敛到最短路**（这里 $x \leftrightarrow z$ 都会收敛到 3，走 y）。 [chapter_5_notes](#)

你做题时的操作套路就是：

1. 写初始：每个节点只知道到自己=0，到邻居=链路 cost，到非邻居= ∞
 2. 交换 DV
 3. 每个节点用 $\min(c + \text{邻居DV})$ 更新
 4. 若有变化就继续发，直到不再变化
-

5-26: Good news travels fast (链路变好时传播快)

课件说“good news travels fast”的场景是：**某条链路代价变小**（或出现更短路）。发生流程是：发现变化的节点立刻更新 DV 并通知邻居；邻居一算发现更短，也立刻更新并再通知下一跳，通常很快全网都“变小”。  chapter_5_notes

课件给了 t0/t1/t2 的叙述：y 先检测到变化并通知，z 更新后再通知，y 再看发现自己的最短路没变就不再发。  chapter_5_notes

你可以把它理解成：min 变小是一种“强信号”，会立刻触发更新与扩散。

5-27: Bad news travels slow: count-to-infinity (链路变差时传播慢)

“bad news”指 **某条链路代价变大** 或某条路断了。课件强调这会导致经典的 **count-to-infinity**：算法需要很多轮才稳定，课件甚至提到一个例子要迭代很多次才停（“44 iterations”）。  chapter_5_notes

为什么会“慢”？

核心原因是：DV 里每个人只看到邻居的“报价”，但看不到邻居这条路到底怎么走。于是可能出现这种互相误导：

- y 的到 x 的路坏了/变贵了，但 y 看到 z 还在说“我到 x 很便宜”（其实 z 那条便宜路可能正是走 y）
- y 就会错误地改成“那我走 z 去 x”
- z 也可能同时觉得“那我走 y 去 x”
- 结果 y 和 z 形成**路由环**，并且它们对“到 x 的距离”会一轮一轮地被迫增大（你看到的就是“数到无穷”）

一句话记忆：坏消息之所以慢，是因为 DV 不携带路径细节，容易出现环路 + 逐步纠正。

 chapter_5_notes

现在停一下，给你提问机会

你想先把哪块问清楚再继续？

- A) 5-24 的距离表：为什么“行=邻居、列=目的地”，以及表项为什么是 $c(x, v) + D_v(dest)$
- B) count-to-infinity：你更想看一个“按轮次递增（6、7、8...）”的具体数值推导
- C) 或者你想直接继续到下一节（课件后面通常会接 LS vs DV 对比 / 再往后进 OSPF/BGP）

来源



继续

已思考 11s >

继续。我们按课件顺序进入 **5-28: Comparison of LS and DV algorithms**（只讲这一页，把“考点对比”讲清楚，然后停下来给你提问）。  chapter_5_notes

5-28: LS (Link State) vs DV (Distance Vector) 对比该怎么背、怎么理解

这页主要从 3 个维度对比：message complexity、convergence、robustness。

 chapter_5_notes

1) Message complexity（消息开销）

LS:

- 本质是“把链路状态信息扩散（flood）到全网”，所以消息量跟 **节点数 n**、**链路数 E** 都相关。课件给的是 **$O(nE)$** 级别。  chapter_5_notes

直觉：每个节点都要把自己的 link-state 广播出去，而且要传遍整个网络。

DV:

- 只跟**邻居**交换距离向量（neighbor-to-neighbor）。  chapter_5_notes

直觉：不会全网泛洪，传播是“一跳一跳扩散”。

2) Speed of convergence (收敛速度 & 可能的问题)

LS:

- 每个节点有全图，跑 Dijkstra (课件也强调算法本身 $O(n^2)$)，总体上比较“可预期”。
📄 chapter_5_notes
- 但可能出现 **oscillations** (比如 cost 跟拥塞相关时，算出来的路由改变会反过来改变拥塞/cost，来回震荡)。 📄 chapter_5_notes

DV:

- 收敛时间 **不固定 (varies)**。 📄 chapter_5_notes
- 两个典型问题：
 - **routing loops (路由环)**
 - **count-to-infinity (数到无穷)**: 坏消息传得慢，可能要很多轮才能稳定 (课件提到过一个例子需要很多 iterations 才稳定)。 📄 chapter_5_notes

你之前已经理解了: DV 只看“邻居报价”，看不到“路径细节”，所以坏消息更容易导致环路/逐步增大。

3) Robustness (鲁棒性: 节点出错会怎样)

LS:

- 如果某个节点乱报 link cost, 影响的是“大家的拓扑数据库”里那一条信息。
- 但每个节点只用这些信息算自己的表 (错误影响相对“局部化”)。 📄 chapter_5_notes

DV:

- 如果某个 DV 节点乱报 path cost, 会直接进入邻居的 min 公式, 被邻居再传播, **错误更容易扩散**。 📄 chapter_5_notes

这一页你要形成的“考试级一句话”

- **LS**: 全网同步拓扑 → 本地算最短路 → 消息多但收敛更可控; 可能震荡。
- **DV**: 只和邻居交换距离 → 收敛不定; 可能环路、count-to-infinity; 错误更易传播。

📄 chapter_5_notes

停一下: 你回答两个小判断 (用来确认你真的会用)

1. 如果网络里链路 cost 会随拥塞动态变化, 你觉得 **LS** 和 **DV** 谁更容易出现“来回改路由”的震荡? 为什么? (提示: 课件把 oscillations 写在 LS 讨论里) 📄 chapter_5_notes

2. “DV 错误更易传播”这句话，你能用一句话解释成因吗？（提示：每个人的表会被别人拿去算 min）  chapter_5_notes

你答完我们就进入下一节 5-30~5-32 Hierarchical routing + AS（为什么要分自治系统、intra-AS vs inter-AS）。

来源

     ...

1. ls更容易震荡（为什么dv不容易震荡） 2. 每个人的表会被别人拿去算min

已思考 15s >

好，先把你第 1 点补完整：为什么课件说 LS 更容易“震荡”，那 DV 为什么相对没那么“典型地”被拿来举震荡例子？

为什么更容易看到 LS 震荡（课件语境）

课件举的“震荡”例子是 **congestion-sensitive routing**：链路 cost = 承载的流量。在这种定义下，LS 的机制是：

- 全网通过 flooding 很快把“新 cost”同步开来；
- 每个路由器几乎同时用同一张拓扑/cost 图跑 Dijkstra，可能集体把流量切到同一条“看起来更便宜”的路；
- 流量一切过去，那条路 cost 又变了 → 再同步 → 再集体切回来.....就形成来回摆动。

 chapter_5_notes

DV 当然也可能在动态 cost 下不稳定，但它的更新是邻居间迭代、异步、传播有滞后，更常见的“典型病”在课件里被强调为 **routing loops / count-to-infinity**（坏消息慢），而不是用它来做“全网同步导致的摆动”示例。

你第 2 点“别人拿我的表去算 min，所以错会扩散”也完全正确。  chapter_5_notes

下面继续下一节：5-30~5-32 Hierarchical routing & AS（为什么要分层/分自治系统）。

 chapter_5_notes

5-30：为什么需要层次化路由

课件给了两个动机：  chapter_5_notes

1. Scale (规模)

如果目的地有几亿个（课件举例 600 million destinations），你不可能：

- 在每台路由器里存下“到每个目的地”的完整路由表；
- 还不停交换这么大的表（会把链路淹没）。  chapter_5_notes

2. Administrative autonomy (管理自治)

Internet 是 “network of networks”，每个网络/运营者都想控制自己网络内部怎么路由，不想被全网统一规则绑死。  chapter_5_notes

课件也点明：我们前面讲 LS/DV 都是假设“网络是扁平的、路由器都一样”，现实不是。

 chapter_5_notes

5-31：把路由器聚合成 AS（自治系统）

解决办法：把路由器分成一个个区域，叫 **Autonomous Systems (AS)**（也叫 domains）。

 chapter_5_notes

于是路由分成两层：  chapter_5_notes

- **Intra-AS routing**：AS 内部路由（同一个 AS 里主机/路由器之间怎么走）
 - 同一个 AS 内所有路由器必须跑同一种 intra-domain 协议
 - 不同 AS 可以用不同的 intra-domain 协议
- **Inter-AS routing**：AS 之间路由（跨 AS 怎么走）
 - 由 **gateway router**（AS 边界上的路由器）参与跨域路由；它既做 inter-domain，也做本域 intra-domain。  chapter_5_notes

5-32：一个路由器的 forwarding table 实际上是谁“共同配置”的？

课件这页很重要：当网络被划分为多个 AS 时，一个路由器的 forwarding table 是 **intra-AS + inter-AS** 一起决定的：  chapter_5_notes

- 去 **AS 内部目的地**：主要由 intra-AS 算法填表
- 去 **AS 外部目的地**：通常需要 inter-AS 决定“应该出哪个网关”，再结合 intra-AS 决定“到这个网关内部怎么走”  chapter_5_notes

停一下给你提问（只围绕这一节）：

1. 你现在能用一句话说清 **gateway router** 的角色吗？

2. “去外部目的地时，为什么既需要 inter-AS 又需要 intra-AS? ”你想让我用 5-32 那张 AS1/AS2/AS3 图走一遍具体路径吗?

来源



1. 决定AS之间怎么走 2. 内部要到网关，外部要到目的as

你两点的核心都对。我把第 1 点再“钉准”一点（更像考试答案），然后我们继续到下一节：
5-33~5-35 Inter-AS：谁用 BGP、为什么是 policy-based。

先把你的第 1 点补精确

gateway router 不是“决定 AS 之间怎么走”的唯一主体（那是 *inter-AS routing protocol* 的工作），它更准确的角色是：

- 它位于 AS 的边界，连接本 AS 和其他 AS；
- 它同时运行：
 - intra-AS 协议（AS 内怎么到达它/从它出发）
 - inter-AS 协议（跨 AS 的可达性与路径选择）
- 所以它是 域内与域间的接口点/出入口。

你第 2 点“内部先到网关、外部再跨域到目的 AS”非常标准。

5-33~5-35：Inter-AS routing（BGP）为什么要“按策略”而不是纯最短路

5-33：Inter-AS 的目标不只是最短路

课件强调：跨 AS 路由和我们前面讲的“最小 cost”不一样，它更像各个 AS 在博弈：谁愿意帮谁转发、走哪条路、是不是收费、是不是避开某些 AS。

所以 inter-AS routing 的核心不是“全网最短”，而是 **policy（策略）**：

- 一个 AS 可能不愿意为某些流量提供 transit
- 一个 AS 可能更偏好走付费更低/商业关系更友好的路径
- 甚至会因为管理原因避开某些路径

5-34: BGP 是 inter-AS 的协议 (“Internet glue”)

课件把 **BGP (Border Gateway Protocol)** 当作 inter-AS routing 的代表协议：它在 AS 之间交换“可达性信息”和“路径属性”，让 gateway router 决定：去某个前缀（目的网络）应该把包交给哪个邻接 AS。

关键点：BGP 的“路由”是对**前缀（prefix）**做的（目的网络/子网），而不是对单个路由器节点。

5-35: 为什么叫 policy-based?

因为 BGP 选路不仅看长度/跳数，还会看各种属性并应用本 AS 的策略（例如商业关系优先级等）。课件要你记住的结论就是：**inter-AS routing = policy dominates**。

现在停一下（确认你掌握“分层”到“BGP策略”的转折）

请你用一句话回答：

1. 为什么 inter-AS 不能简单用“全网最短路”（像 Dijkstra 那样）解决？
2. BGP 在 AS 之间主要交换的是什么类型的信息（别背细节，抓“可达性 + 路径/属性 + 前缀”这个层级即可）



你漏掉了rip和ospf

已思考 39s >

好，你说得对：5-33 那页把 RIP 和 OSPF 都列成了最常见的 intra-AS (IGP) 协议，我们刚才讲 DV/LS 原理时把“协议实例”这块跳过了。我们现在就按课件的 5.3 节把它补上（这一节我只讲 **IGP: RIP + OSPF**，不往 BGP 走）。  chapter_5_notes

5.3 Intra-AS Routing (IGP): RIP vs OSPF

课件先给出定位：intra-AS routing 也叫 IGP，常见协议：RIP、OSPF（还有 Cisco 的 IGRP）  chapter_5_notes

RIP (Routing Information Protocol): 把你刚学的 DV “落地成协议”

把 RIP 直接当成“DV 算法在真实网络里的一个经典实现”就行：

- **每个路由器维护一个 distance vector**：对“AS 内所有目的网络/路由器”的距离估计（这就是你之前答的：长度更接近 20 而不是 3）。其更新逻辑就是 Bellman-Ford 那句：

$$D_x(y) \leftarrow \min_v \{c(x, v) + D_v(y)\}$$

（和课件 5-21/5-22 的 DV 描述完全一致）  chapter_5_notes

- **度量 (metric) 通常是 hop count (跳数)**，并且有个硬上限：**最大 15 hops**；16 代表不可达，所以 RIP 适合小/中型网络，不适合很大规模。 RFC 编辑器 +1
- **周期性把整张路由表发给邻居**（典型周期是 30 秒），用的是 **UDP 520 端口**（RIP v1/v2）。 Juniper Networks

这也解释了你问的“为什么 DV 不容易震荡”：DV 的信息传播是邻居到邻居的局部扩散，不会像 LS 那样一变动就让全网每台路由器都立刻重算；但代价是**收敛可能慢**、还会有 **routing loop / count-to-infinity**（课件 5-27 明说了 bad news travels slow）。

 chapter_5_notes

你可以用一句话记：**RIP = DV + hop-count + periodic full-table updates**（小网络好用，大网络吃力）。 RFC 编辑器 +2

OSPF (Open Shortest Path First)：把你刚学的 LS + Dijkstra “落地成协议”

课件对 OSPF 的三步法很明确（5-34）  chapter_5_notes :

1. **Flood link-state 信息**（LSA 在域内泛洪）
2. **每台路由器构建“整个 AS 的拓扑图”**
3. **每台路由器在自己的拓扑图上跑 Dijkstra**，得到“以自己为根”的 shortest-path tree，然后生成自己的 forwarding table  chapter_5_notes

补两个很关键的“协议层面落地细节”（课件也暗示了）：

- OSPF 的报文是 **直接跑在 IP 上**（不是 TCP/UDP）；在 IP 里有专门的协议号 **89**。
 chapter_5_notes NetLab +1
- **分层 OSPF (two-level hierarchy)**：多个 area + 一个 backbone；LSA 只在 area 内泛洪，跨 area 通过 **ABR 汇总**，并有 **ASBR** 连接到其它路由域（比如去接 BGP 或别的域）。  chapter_5_notes

一句话记：**OSPF = LS + flooding + 全图 + Dijkstra + area/backbone 做规模化**。

 chapter_5_notes

给你两个“确认理解”的小问题（你一句话答就行）

1. 为什么 RIP 的 hop-count 上限 (15) 会直接限制网络规模? (用“路径长度/可达性”的角度答) RFC 编辑器 +1
2. OSPF 为什么要引入 area/backbone (ABR 汇总)? 它解决的是“哪一种开销”变得太大? 📄 chapter_5_notes

你答完我就按课件顺序继续往下讲 5.4 BGP (eBGP/iBGP、path vector、route selection、hot potato)。

来源

📄 🍌 🗨️ ⬆️ ↺️ ... 🌐 IPE 🌐 来源

1. 最短路径也可能比上限大, 所以大规模网络不适合
2. 节点太多时表太大

已思考 29s >

你这两题都对, 我帮你把“考试用的更精确表述”补一刀:

1. RIP hop-count 上限

是的: 因为 RIP 的 metric 是 hop count, 且最大可用值是 15, 16 视为不可达, 所以网络直径一大 (最短路 hop 数可能 >15) 就直接用不了/会被当成 unreachable。 维基百科

2. OSPF 为啥要 area/backbone

你说的“节点太多表太大”是对的, 更精确: OSPF 的 link-state 需要泛洪 LSA + 维护 LSDB (拓扑数据库) + 跑 Dijkstra, 规模大时这些开销都爆; 于是用 two-level hierarchy: 多个 area + 一个 backbone, 把 LSA 的传播限制在 area 内, 跨 area 通过 ABR 做汇总/通告。 📄 chapter_5_notes

下面我们按课件继续 5.4: BGP (inter-AS routing), 我只讲一节: BGP 的“到底在交换什么 + eBGP/iBGP 怎么分工 + path vector/属性”, 讲完停下来让你问。 📄 chapter_5_notes

5-38~5-42: BGP 在干什么

1) inter-AS 要解决的任务

AS1 里某路由器收到一个“目的地在 AS 外”的包: 问题不是“怎么到目的地”, 而是先选哪个 gateway (出口)。为此 AS1 必须

- 学到: 哪些目的前缀通过 AS2 可达、哪些通过 AS3 可达

- 再把这些“可达性信息”传到 AS1 内所有路由器
这就是 inter-AS routing 的工作。  chapter_5_notes

2) eBGP vs iBGP: 一个“拿外部信息”, 一个“在 AS 内扩散”

课件的定义很清楚:  chapter_5_notes

- **eBGP**: 从相邻 AS 获取“某前缀可达”的信息 (reachability)
- **iBGP**: 把这些 reachability 信息在 **本 AS 内部**传播给所有路由器
并且 gateway routers 往往两种都跑。  chapter_5_notes

3) BGP 是 path vector: 广告“到前缀的路径 (经过哪些 AS)”

BGP 会“广告”到某个 **destination network prefix** 的路径; 课件例子: AS3 的 3a 给 AS2 的 2c 发 AS3, X, 含义是**“我 (AS3) 承诺能把去 X 的包转发过去”**。  chapter_5_notes
同时, BGP peers 之间的会话是跑在 **TCP 连接**上的。  chapter_5_notes

4) BGP route = prefix + attributes (两个最重要的属性)

课件点名了两个属性:  chapter_5_notes

- **AS-PATH**: 这条前缀通告一路经过的 AS 列表 (用来避免环、也常用于选路)
- **NEXT-HOP**: 进入这条 AS-PATH 的“下一跳路由器接口 IP” (转发时要用)

并且强调: **policy-based routing**

- import policy: 收不收这条路 (比如“绝不经过某 AS”)
- export policy: 要不要把这条路再广告给其他邻居 AS
这解释了“inter-AS 不一定追求最短/最快”, 而是**策略优先**。  chapter_5_notes

现在停一下: 两道超关键确认题

1. 在课件里, 为什么说 BGP 是 “path vector”? 它和 DV 最大的区别用一句话怎么说?
 chapter_5_notes
2. 你能用自己的话解释 **AS-PATH** 和 **NEXT-HOP** 分别解决什么问题吗?
 chapter_5_notes

来源

     ...  来源

1. bgp是走的路记下来, 没走的不记, dv是一个全表2. 不

懂。问题：ibgp和内部的rip或者ospf有什么联系

已思考 10s >

你第 1 句抓到重点了：**BGP 会把“经过哪些 AS”这条路写进来 (AS-PATH)**，这就是 “path vector”。  chapter_5_notes

但我帮你把“DV 是全表”这点纠正得更精确一点：

- **DV (距离向量)**：每个路由器对“每个目的地”的距离估计 $D_x(y)$ ，并和邻居交换这条“距离向量”。它不告诉你“路径长什么样”，只告诉你“代价多大”。  chapter_5_notes
- **BGP (path vector)**：对每个前缀 (prefix) 通告一条“路由 = prefix + 属性”，其中关键属性包括 **AS-PATH** (路径经过的 AS 列表) 和 **NEXT-HOP**。

你说 “AS-PATH / NEXT-HOP 不懂”——一口气讲清

AS-PATH 是干嘛的？

AS-PATH = 这条前缀通告一路经过的 AS 序列。  chapter_5_notes

它至少解决两件事：

1. **避免环路**：如果我在 AS-PATH 里看到“我自己这个 AS”，那说明绕回来了，直接拒掉（这是 path vector 相比 DV 很重要的改进）。
2. **参与选路**：课件的 route selection 里明确有一条：**shortest AS-PATH**。

 chapter_5_notes

NEXT-HOP 是干嘛的？

NEXT-HOP = “从我这里发出去，要先把包交给谁（哪个路由器接口 IP）才能进入这条 AS-PATH”。  chapter_5_notes

注意：它不是“AS 层面的下一跳”，而是数据平面真正要用的下一跳接口 IP（否则你有 AS-PATH 也没法转发）。

你问：iBGP 和内部 RIP/OSPF (IGP) 有什么联系？

一句话：**iBGP 负责“告诉你外面的路有哪些”，IGP (OSPF/RIP) 负责“在 AS 内把你送到那个出口/下一跳”。**两者是配套的，不是替代。

把它拆成 3 个非常具体的联系点：

1. 分工不同

- **iBGP**：把外部可达性（reachability）传播到 AS 内所有路由器。
- **OSPF/RIP**：AS 内部最短路/可达性（到各内部子网、到各网关路由器）。

📄 chapter_5_notes

2. BGP 选路里会直接用到 IGP 的“内部代价”

课件 route selection 第 3 条：**closest NEXT-HOP router: hot potato routing**。

📄 chapter_5_notes

“closest”这个“近”，就是用 **OSPF/RIP** 算出来的 **intra-domain cost** 来衡量（课件甚至在 hot potato 的图上标了 OSPF link weights）。📄 chapter_5_notes

所以：**iBGP** 给你多个出口可选，**IGP** 决定哪个出口在 AS 内更便宜，于是你‘热土豆’把流量尽快丢出本 AS。

3. 最终 forwarding table 是 inter-AS + intra-AS 共同配置出来的

课件明确说：forwarding table 对外部目的地的条目，是 **inter-AS & intra-AS** 一起作用的：先由 inter-AS 决定“走哪个出口/哪条外部路”，再由 intra-AS 决定“怎么到那个出口”。📄 chapter_5_notes

如果你理解到这儿没问题，我们下一节就顺着课件往下：**BGP path advertisement 的传播（eBGP 学到 → iBGP 扩散 → 再 eBGP 广告） + BGP route selection + hot potato**。

来源

📄 🍌 🗨️ ⬆️ ↻️ ...

next-hop是一个ip: port吗

不是“IP:port”。

在 BGP 里 **NEXT-HOP** 是一个 **IP 地址**（下一跳路由器的接口 IP），供数据平面转发时当作“下一跳”用。📄 chapter_5_notes

- **BGP 控制平面**的邻居会话本身是跑在 **TCP** 上（所以确实有端口），但那是建立 BGP session 用的；
- **NEXT-HOP 属性**里存的不是端口，只是下一跳的 **IP**。📄 chapter_5_notes

来源

📄 🍌 🗨️ ⬆️ ↻️ ...

然后as-path是什么东西

AS-PATH 是 BGP 路由通告里的一个属性，意思是：

“这条到某个目的前缀（prefix）的路由，在跨域转发时会依次经过哪些 AS（自治系统）。”

📄 chapter_5_notes

你可以把它想成一串 AS 号的列表，比如：

AS3 AS2 AS1 （只是示意）

它主要有两大用处：

1. **防环路**：如果我收到的 AS-PATH 里已经出现了“我自己的 AS”，说明这条通告绕回来了，我就拒绝它。
2. **参与选路**：BGP 选路规则里通常会偏好 **AS-PATH 更短** 的路由（跳过更少的 AS）。

📄 chapter_5_notes

来源

📄 🍌 🗨️ ⬆️ ↺️ ...

所以有可能的事nexthop是as2的ip地址吗，如果当前是在as1的时候

对，完全有可能。

当你在 **AS1** 里学到一条“去某个前缀 X”的 BGP 路由时，那个路由的 **NEXT-HOP** 往往就是：

- **AS2 边界路由器（gateway router）** 在 **AS1-AS2** 这条互联链路路上的接口 IP（也就是“你第一跳要交给的那台路由器的接口地址”）。 📄 chapter_5_notes

一个常见流程是：

1. **AS1 的边界路由器**通过 **eBGP** 从 AS2 学到前缀 X 的通告，NEXT-HOP 就指向 AS2 侧邻居的接口 IP。

2. 这条路由再通过 **iBGP** 传播到 AS1 内部其他路由器时，**NEXT-HOP 通常保持不变**（除非配置了 `next-hop-self` 之类，让它改成 AS1 边界路由器自己的 IP）。

所以在 AS1 内部的普通路由器看来：

“去 X 的下一跳”可能就是一个 **不属于 AS1 的 IP（AS2 边界接口 IP）**，但它会靠 **IGP（OSPF/RIP）** 先把包送到能到达那个 next-hop 的出口/边界路由器，再出 AS1。这个机制也对应课件里讲的 inter-AS 与 intra-AS 共同决定转发、以及 hot-potato 会用到 IGP 距离。

来源



继续

已思考 9s >

好，继续按课件把 **BGP 这条线** 讲完：**路径如何传播（path advertisement）→ 路由选择规则（route selection）→ hot potato（热土豆）**。我只讲这一节，讲完停下来让你提问。

chapter_5_notes

5-44~5-45：BGP path advertisement（路由是怎么“传开”的）

看课件那张 AS1/AS2/AS3 + 目的前缀 X 的图： chapter_5_notes

1. eBGP 学到外部路

AS3 的边界路由器 3a 向 AS2 的 2c 通告：AS3，X（意思是“去 X 这条路经过 AS3”）。 chapter_5_notes

2. AS2 内部用 iBGP 扩散

AS2 的 2c 接受后，把这条可达性通过 **iBGP** 传播给 AS2 内其它路由器（“让整个 AS2 都知道”）。 chapter_5_notes

3. 再用 eBGP 向别的 AS 广告（加上自己 AS 号）

AS2 的另一个边界路由器 2a 再对 AS1 的 1c 通告：AS2，AS3，X（把自己 AS2 prepend 到 AS-PATH 里）。 chapter_5_notes

于是 AS1 的 1c 可能同时学到多条去 X 的路（例如从 AS2 来的、也可能直接从 AS3 来的），接下来就要做 **route selection**。 chapter_5_notes

5-46: BGP route selection (选路顺序)

课件给了一个非常“考试友好”的选择顺序：  chapter_5_notes

1. **local preference (本地偏好)**: 策略决定 (policy decision)
2. **shortest AS-PATH**: AS 跳数更少优先
3. **closest NEXT-HOP router: hot potato routing** (选“离 next-hop 最近”的出口)
4. 其它额外条件 (课件没展开)

注意: 这就是为什么 BGP 说自己是 **policy-based**: 第一条直接就是“策略优先”。

 chapter_5_notes

5-47: Hot Potato Routing (热土豆到底在干嘛)

课件这张图的核心句是: “选择 intra-domain cost 最小的本地网关, 把流量尽快丢出本 AS; 别太关心域间成本!”  chapter_5_notes

在图里:

- 路由器 2d (在 AS2 内) 通过 iBGP 知道: 去 X 可以走 **2a** 或 **2c** 两个出口。
 chapter_5_notes
- hot potato 的决策依据不是 “哪个 AS-PATH 更短” (那是上一步), 而是: 从 2d 到 2a / 2c 的 AS 内部代价谁更小 (图里用 OSPF link weights 表示)。
- 所以 2d 可能会选 2a ——即便选 2a 会导致跨域 AS hops 更多, 仍然“先把包丢出去”。

 chapter_5_notes

这也回答了你之前问的 “iBGP 和 OSPF/RIP 有啥联系”: **BGP** 给你多个出口可选, **OSPF/RIP** 给你“到各出口的内部距离”, hot potato 用的就是这个内部距离。

现在停一下: 你回答两个小题 (很短)

1. 在课件的 route selection 顺序里, 为什么 **local preference** 放在 **AS-PATH** 前面?
(用 “policy dominates” 那句话解释)  chapter_5_notes
2. hot potato 里说的 “closest NEXT-HOP router”, 这个 “closest” 是靠谁算出来的:
iBGP 还是 OSPF/RIP? 为什么?  chapter_5_notes

来源



1. policy dominates 2. ospf/rip

对，两个都完全正确。  chapter_5_notes

我们再顺着课件往下走到 **5.5 SDN 控制平面**（把这一章最后一个大块讲开）：**为什么要 SDN、controller 做什么、OpenFlow 的“match-action”直觉**。我这次只讲 SDN 的“概念/架构”这一节，讲完停下来给你提问。

5.5 SDN Control Plane: 从“每台路由器各算各的”到“逻辑集中控制”

1) 回忆 5.1: 两种控制平面

课件一开始就说过：传统是 **per-router control**，SDN 是 **logically centralized control**。SDN 这一节就是把“逻辑集中”讲具体：**谁集中？集中干什么？怎么下发？**

2) SDN 的核心思想：control plane 和 data plane 分离

- **data plane (转发平面)**：交换机/路由器做的事很“机械”——按规则把包从某个端口转到另一个端口 (match→action)。
- **control plane (控制平面)**：决定这些规则应该是什么（全局视角，统一算、统一下发）。

课件里 SDN 的图一般会画成：

- 下方是一堆 **switches**（只做 forwarding）
- 上方是一个（逻辑上）中心的 **SDN controller**
- 中间有一个南向接口（southbound API），经典例子是 **OpenFlow**。

3) Controller 具体在做什么（你可以把它当“大脑 + 操作系统”）

概念上 controller 负责三类事：

1. **收集网络状态**（拓扑、链路利用率、故障、交换机端口信息等）
2. **运行控制应用（control apps）**
 - 例如：路由（算路径）、流量工程（TE）、访问控制（ACL）、负载均衡、防火墙策略等
3. **把结果下发成转发规则到各个交换机的 flow table 里**（让 data plane 执行）。

4) OpenFlow 的直觉：match-action flow table

你现在不用背 OpenFlow 报文细节，只要抓住这个：交换机里有一张/多张 **flow table**，每条规则长这样：

- **match**：匹配哪些包（可以按入端口、MAC、IP、TCP/UDP 端口等字段）
- **action**：匹配到后干什么（转发到某端口、丢弃、改写 header、发给 controller...）
- （通常还有 counters、priority、timeout 之类）

这和我们前面说的“forwarding table 只按目的 IP 前缀选下一跳”相比，SDN 的规则更一般、更可编程。

现在停一下：你问 / 我问你两道确认题

1. 用你自己的话说：SDN 相比传统路由，“集中控制”到底集中的是哪部分？
（forwarding 还是 routing/策略？）
2. match-action 表达能力更强，这意味着 SDN 更容易实现哪类功能：
 - A 只做最短路选路
 - B 做访问控制/按应用分流/负载均衡这类策略

你答完我们再继续把课件后面关于 **controller 的优势/挑战**、以及 **为什么说是 logically centralized** 这些点补齐。

来源



1. routing, 2. b

已思考 26s >

对，1= routing（控制平面算路/策略），2= B，完全到位。  chapter_5_notes

那我们继续把这节课件里关于 “logically centralized control plane (SDN)” 的关键点讲清楚（只讲这一小节，讲完停）。  chapter_5_notes

5.1/5.5 里 SDN 的核心：logically centralized ≠ physically centralized

课件把控制平面结构分两类：

- **per-router control (传统)**：每台路由器里都有“routing algorithm 的组件”，彼此交互，算出各自的 forwarding table。  chapter_5_notes
- **logically centralized control (SDN)**：有一个“distinct (通常远程) 的 controller”，它和每台路由器/交换机上的控制代理 (**control agent, CA**) 交互，然后计算并下发这些设备需要用的 forwarding table。  chapter_5_notes

这里最容易混淆的是“集中”到底是什么意思：

1) “logically centralized” 强调的是“决策逻辑集中”

也就是：路径/策略怎么定，主要在 controller 的统一视角里做（像一个大脑）。

但这不等于 controller 物理上只能有一台机器。

2) 物理实现通常会“分布式部署”，但“看起来像一个大脑”

现实系统里为了可靠性/性能，controller 往往是多副本、多实例；对外仍然表现得像“一个统一控制平面”（所以叫 *logically centralized*）。

课件用词也留了口子：controller 是 distinct，且 typically remote。  chapter_5_notes

3) CA 的角色：把 controller 的决定落到设备上

你可以把 CA 当成“本地执行/对接层”：

- controller 负责算（算路由/策略/表项）
- CA 负责跟 controller 通信、把表装进本机的数据平面，让转发照表执行。

 chapter_5_notes

三 ChatGPT 5.2 Thinking ▾

   ...

1. 你觉得 **logically centralized** 还可以引申，**logically** 是时刻提醒无哪种误解：

2. 如果 controller 临时不可达，**数据平面**会立刻不能转发吗？（你按直觉答就行，我们下一步再把“为什么”讲严谨）

来源

     ...

继续，tcmp

已停止思考 >

继续, icmp

已思考 32s >

好，我们继续到 **5.6 ICMP**（以及它和 **ping / traceroute** 的关系）。  chapter_5_notes

5-50: ICMP 是什么？它在 IP 的“上面”，但装在 IP 里走

课件的三句话你要背熟：  chapter_5_notes

1. **ICMP 用途**：主机和路由器用它来交换网络层信息

- **错误报告**：比如目的网络/主机/端口/协议不可达
- **echo request/reply**：也就是 ping 用的那对报文  chapter_5_notes

2. “**network-layer above IP**”的意思：

ICMP 是网络层控制协议，但 **ICMP 报文是被封装在 IP datagram 里传输的**（所以它不是单独的一层链路协议）。  chapter_5_notes

3. **ICMP 报文格式核心**：

ICMP 报文里有 **type + code**，并且（在出错类报文中）会带上“触发错误的那个 IP datagram 的前 8 字节”等信息，方便源端定位是哪一个包出的问题。  chapter_5_notes

你现在要会看这张表（type/code）

最常考的几条：  chapter_5_notes

- **type 8 code 0**：echo request（ping 发出去的）
- **type 0 code 0**：echo reply（ping 回来的）
- **type 3 (Destination unreachable)**：下面 code 区分原因
 - code 0：network unreachable
 - code 1：host unreachable
 - code 3：port unreachable（traceroute 停止就靠它）
- **type 11 code 0**：TTL expired（traceroute 每一跳回来的就是它）
- **type 12**：bad IP header  chapter_5_notes

5-53: Traceroute 为什么和 ICMP 强绑定？

课件给的“设计 traceroute 的套路”是这样的：  chapter_5_notes

1. 源主机向目的地发送一系列 **UDP segments** (注意：不是 ICMP 发过去的)，但用一个几乎不可能被应用监听的端口号。  chapter_5_notes
2. 第一组包 **TTL=1**，第二组 **TTL=2**，依次递增。  chapter_5_notes
3. 当 **TTL=i** 的包到达路径上的第 **i** 个路由器：
 - 路由器发现 TTL 变成 0，会丢弃该包
 - 并回给源端一个 **ICMP Time Exceeded (type 11, code 0)**，里面带路由器名字/IP
 - 源端用“发出到收到 ICMP 的时间差”记录这一跳 RTT
4. 什么时候停止？
当某一组 UDP 包真的到达目的主机时，因为端口号很怪，目的主机会回 **ICMP Port Unreachable (type 3, code 3)**，源端收到后就停止。  chapter_5_notes

课件里 traceroute 输出一行三次时间 (3 probes)，**"*"**表示某次 probe 没收到回复 (可能丢了或路由器不回)。  chapter_5_notes

停一下，给你两个很小的自检题 (你一句话答就行)：

1. traceroute 为什么要用 **UDP + 不太可能的端口**，而不是随便发 UDP？它靠目的端回哪个 ICMP 来“结束”？  chapter_5_notes
2. 每一跳路由器回给你的 ICMP 是哪个 type/code？它为什么能暴露“第 i 跳是谁”？
 chapter_5_notes

来源



1. 不建立连接，而且是sender自发2. ?

你第 1 点里“不建立连接/自发探测”这个方向对，但还差**关键机制**。我直接把两题的正确答案讲清 (这段非常考点)，然后你再用自己的话复述一遍就行。

1) traceroute 为什么用 UDP + “奇怪端口”？靠哪个 ICMP 结束？

traceroute 经典做法是：向目的主机发 UDP 报文，但目的端口选一个几乎肯定没人监听的端口。这样当探测包最终真的到达目的主机时：

- 目的主机发现“这个 UDP 端口没人开”
- 就回一个 **ICMP Destination Unreachable – Port Unreachable (type 3, code 3)**

- 源主机收到这个 **type 3 code 3** 就知道“已经到目的地了”，所以停止。

所以“奇怪端口”不是为了“不建立连接”，而是为了保证最终能触发一个确定的 ICMP 作为终止信号。

2) 每一跳路由器回来的 ICMP 是哪个 type/code? 为什么能知道第 i 跳是谁?

当你发 TTL=1/2/3... 的 UDP 探测包时，第 i 个路由器会把 TTL 减到 0 并丢包，然后回：

- **ICMP Time Exceeded (type 11, code 0)**

为什么能知道“第 i 跳是谁”?

- 因为这条 ICMP 是由那个路由器自己发回来的，源端看到回包的源 IP（以及 **traceroute** 输出的 **router 名称/IP**），就知道这一跳的路由器身份；同时用往返时间得到这一跳 RTT。

你现在用一句话复述一下：

- “中间路由器回的是什么 ICMP? ”
- “到达目的主机时回的是什么 ICMP? ”



你复述一下吧

当然可以，我按你要的“复述版”来一遍（就是你考试时能直接写出来的那种）：

- traceroute 会向目的主机发送一串 UDP 探测报文，TTL 从 1 开始逐步增加。每当某个探测报文在第 i 跳路由器处 TTL 变成 0 被丢弃时，该路由器会回一个 **ICMP Time Exceeded (type 11, code 0)**，源主机就用这个回包的源 IP/名称识别“第 i 跳是谁”，并用往返时间算这一跳 RTT。
- 当探测报文最终到达目的主机时，由于目的端口选的是一个通常没人监听的高端口，目的主机会回一个 **ICMP Destination Unreachable – Port Unreachable (type 3, code 3)**，源主机收到这个回包就知道“已经到目的地”，于是 traceroute 结束。



